



Birla Institute of Technology & Science, Pilani

Work-Integrated Learning Programmes Division
MTech in Artificial Intelligence and Machine Learning
S2_2025-2026, AIMLCZG557/AECLZG557

Assignment 2 – PS1 - Two-Player Sudoku Completion - [Weightage 13%]

Read through this entire document very carefully before you start!

Problem Statement

A partially completed 9×9 Sudoku puzzle is given. Two players, Player 1 and Player 2, take turns filling one empty cell with a number from 1 to 9. A move is considered valid if the inserted number satisfies all Sudoku rules:

- The number must be unique in its row.
- The number must be unique in its column.
- The number must be unique in its corresponding 3×3 subgrid.

Scoring Rules

- A player earns points equal to the value of the correctly placed number.
 - Example: Placing 8 correctly earns 8 points.
- An invalid move is not allowed and the player receives 0 points for that turn.
- The game continues until all empty cells are filled.

Player 1 (MAX) aims to maximize the final score, while Player 2 (MIN) aims to minimize Player 1's advantage by selecting the best possible valid moves.

Implement the Minimax algorithm to determine the optimal move for the current player. The algorithm should generate the game tree, evaluate possible future moves, and select the move that produces the best outcome assuming both players play optimally.

If multiple moves have the same evaluation value, choose the move with the smallest row number, followed by the smallest column number.

Finally, print the selected move, the updated Sudoku board, and the scores of both players.

Input Format

- Nine lines representing the Sudoku board.
- Empty cells are represented by ..
- The next line contains the current player (P1 or P2).

Output Format

Best Move:

Row = <row_number>

Column = <column_number>

Number = <value>

Player 1 Score = <score>

Player 2 Score = <score>

Updated Sudoku Board:

<updated board>

Constraints

- Sudoku size = 9×9
- Only valid Sudoku moves are allowed.
- Assume both players play optimally.
- Implement the Minimax algorithm without Alpha-Beta pruning.

Sample Input

TEST CASE 1

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			
				8			7	9

P1

TEST CASE 2

5	3	4	6	7	8	9	1	
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	7	7	9

P2

Sample Output

TEST CASE 1

Best Move:

Row = 1

Column = 3

Number = 4

Player 1 Score = 4

Player 2 Score = 0

Updated Sudoku Board

5	3	4		7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			
				8			7	9

TEST CASE 2

Best Move:

Row = 1

Column = 9

Number = 2

Player 1 Score = 0

Player 2 Score = 2

Updated Sudoku Board

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	7	7	9

Note that the input/output data shown here is only for understanding and testing, the actual file used for evaluation will be different.

Display the output in outputPSXX.txt.

Deliverables:

- PDF document **designPSXX_<group id>.pdf** detailing your solution.
- **[Group id]_Contribution.xlsx** mentioning the contribution of each student in terms of percentage of work done. Columns must be “Student Registration Number”, “Name”, “Percentage of contribution out of 100%”. If a student did not contribute at all, it will be 0%, if all contributed then 100% for all.
- **inputPSXX.txt** file used for testing
- **outputPSXX.txt** file generated while testing
- .py file containing the python code. Do not fragment your code into multiple files
- Zip all of the above files including the design document in a folder with the name:
 - **[Group id]_A2_PSXX.zip** and submit the zipped file.
 - Group Id should be given as **Gxxx** where **xxx** is your group number. For example, if your group is 26, then you will enter **G026** as your group id.

Instructions:

- It is compulsory to make use of the data structure(s) / algorithms mentioned in the problem statement.
- Ensure that all data structure insert and delete operations throw appropriate messages when their capacity is empty or full. Also ensure basic error handling is implemented.

- For the purposes of testing, you may implement some functions to print the data structures or other test data. But all such functions must be commented before submission.
- Make sure that you read, understand, and follow all the instructions
- Ensure that the input, prompt and output file guidelines are adhered to. Deviations from the mentioned formats will not be entertained.
- The input, prompt and output samples shown here are only a representation of the syntax to be used. Actual files used to evaluate the submissions will be different. Hence, do not hard code any values into the code.
- Please note that the design document must include:
 - One alternate way of modeling the problem with the performance implications.
 - Writing a good technical report and well documented code is an art. Your report cannot exceed 4 pages. Your code must be modular and quite well documented.
 - You may ask queries [in this dedicated discussion section](#). Beware that only hints will be provided and queries asked **in other channels like MS Teams, emails, Taxila inbox will not be responded to.**

Deadline

1. The strict deadline for submission of the assignment is **17th Aug 2026 11:55 PM IST (Monday)**.
2. The deadline has been set considering extra days from the regular duration in order to accommodate any challenges you might face. No further extensions will be entertained.
3. Late submissions will be evaluated as below. Submissions made between:
 - a. 17th Aug 2026 11:56PM to 18th Aug 2026 11:55PM → Deduct 2M for late submission and evaluation for 10 marks.
 - b. 18th Aug 2026 11:56PM to 19th June 2026 11:55PM → Deduct 6M for late submission and evaluation for 6marks.
 - c. Beyond 19th Aug 2026 11:55PM → No evaluations, 0 marks will be awarded.

How to submit

- This is a group assignment.
- Each group has to make one submission (only one, no resubmission) of solutions.
- Each group should zip all the deliverables in one zip file and name the zipped file as [Group id] _A2_PSXX.zip and submit the zipped file.
- Group Id should be given as Gxxx where xxx is your group number. For example, if your group is 26, then you will enter G026 as your group id.
- Assignments should be submitted via Taxila > Assignment section. Assignments submitted via other means like email etc. will not be graded.

Evaluation

- The assignment carries **13 Marks**.

- Grading will depend on:
- Fully executable code with all functionality working as expected
 - Well-structured and commented code
 - Accuracy of the design document.
 - Every bug in the functionality will have negative marking.
 - Marks will be deducted if your program fails to read the input file used for evaluation due to change / deviation from the required syntax.
- We encourage students to take the upcoming assignments and examinations seriously and submit only original work. Please note that plagiarism in assignments will be taken seriously. Refer to the detailed policy [here](#).
- Source code files which contain compilation errors will get at most 25% of the value of that question.

Readings

Artificial Intelligence: A Modern Approach Textbook by Peter Norvig and Stuart J. Russell, 4th Edition.